Cryptography and Network Security

HTTP Request Integrity Verification Using HMAC

Name: Shrihari V

Reg. No: 23BCI0083

1. Introduction

Web applications communicate with servers using HTTP requests and responses. During this communication, parameters such as account numbers and transaction amounts are transmitted between the client and the server. If these requests are intercepted and modified by an attacker, it can lead to unauthorized actions such as altering transaction values.

This project demonstrates how HTTP request parameters can be intercepted and tampered with using Burp Suite, and how HMAC-SHA256 (Hash-based Message Authentication Code) can be used to detect such tampering at the server side. The defence mechanism was implemented as a Flask server that verifies the HMAC of every incoming request before processing it.

2. Tools Used

- Python 3 — Flask, hmac, hashlib, requests
- Burp Suite Community Edition — HTTP proxy for request interception
- Postman — HTTP request generation and testing

3. Attack Demonstration — Burp Suite

Burp Suite was configured as a proxy to intercept HTTP POST requests sent from Postman. The intercepted request revealed the full HTTP structure including headers and parameters, which could be freely modified before forwarding to the server.

A POST request was sent to postman-echo.com with parameters transfer=5000 and account=B. The transfer value was then modified to transfer=7000 in Burp Suite before forwarding, and the server accepted the tampered value — demonstrating the vulnerability when no integrity check is in place.

When HMAC was added to the request and the transfer value was again modified (5000 → 9000), the HMAC remained unchanged. The server now detected the mismatch and rejected the request, demonstrating effective integrity protection.

4. Implementation

4.1 server.py — Flask Server with HMAC Verification

The Flask server exposes two endpoints. The /transfer endpoint verifies the HMAC of each incoming request and rejects tampered requests with a 403 response. The /transfer-no-hmac endpoint has no integrity check and accepts any request blindly — included to contrast the vulnerability.

```
from flask import Flask, request, jsonify
import hmac
import hashlib

app = Flask(__name__)
```

```python
SECRET_KEY = "Shrihari_23BCI0083"


def generate_hmac(message: str) -> str:
    """Generate HMAC-SHA256 for a given message using the shared secret key."""
    return hmac.new(
        SECRET_KEY.encode(),
        message.encode(),
        hashlib.sha256
    ).hexdigest()


def verify_hmac(message: str, provided_hmac: str) -> bool:
    expected_hmac = generate_hmac(message)
    # Use hmac.compare_digest to prevent timing attacks
    return hmac.compare_digest(expected_hmac, provided_hmac)


@app.route("/transfer", methods=["POST"])
def transfer():
    transfer_amount = request.args.get("transfer")
    account = request.args.get("account")
    provided_hmac = request.args.get("hmac")


    if not all([transfer_amount, account, provided_hmac]):
        return jsonify({
            "status": "error",
            "message": "Missing parameters. Required: transfer, account, hmac"
        }), 400
    message = f"transfer={transfer_amount}&account={account}"
    if verify_hmac(message, provided_hmac):
        return jsonify({
            "status": "success",
            "message": "Request accepted. HMAC verified — data integrity confirmed.",
            "data": {
                "transfer": transfer_amount,
                "account": account
            }
        }), 200
    else:
        return jsonify({
            "status": "rejected",
            "message": "Request rejected. HMAC mismatch — possible tampering
detected.",
```

```
            "provided_hmac": provided_hmac,
            "expected_hmac":
generate_hmac(f"transfer={transfer_amount}&account={account}")
        }), 403


@app.route("/transfer-no-hmac", methods=["POST"])
def transfer_no_hmac():
    """
    Insecure endpoint with NO integrity check.
    Demonstrates the vulnerability: any tampered value is accepted blindly.
    """
    transfer_amount = request.args.get("transfer")
    account = request.args.get("account")

    if not all([transfer_amount, account]):
        return jsonify({
            "status": "error",
            "message": "Missing parameters."
        }), 400

    return jsonify({
        "status": "success",
        "message": "Request accepted (NO integrity check — vulnerable endpoint).",
        "data": {
            "transfer": transfer_amount,
            "account": account
        }
    }), 200


if __name__ == "__main__":
    print("Starting HMAC verification server on http://127.0.0.1:5000")
    print(f"Secret key in use: {SECRET_KEY}")
    app.run(debug=True)
```

4.2 client_demo.py — Demo Script

The client demo script runs four scenarios automatically, showing HMAC generation, legitimate request acceptance, tampered request rejection, and the insecure endpoint accepting tampered values blindly.

```
import hmac
import hashlib
import requests
```

```python
BASE_URL = "http://127.0.0.1:5000"
SECRET_KEY = "Shrihari_23BCI0083"

def generate_hmac(message: str) -> str:
    """Generate HMAC-SHA256 for a message using the shared secret key."""
    return hmac.new(
        SECRET_KEY.encode(),
        message.encode(),
        hashlib.sha256
    ).hexdigest()


def print_separator(title: str):
    print("\n" + "=" * 60)
    print(f"  {title}")
    print("=" * 60)

# Demo 1
def demo_legitimate_request():
    print_separator("DEMO 1: Legitimate Request")

    transfer = "5000"
    account = "B"
    message = f"transfer={transfer}&account={account}"
    hmac_value = generate_hmac(message)

    print(f"  Message   : {message}")
    print(f"  HMAC      : {hmac_value}")

    response = requests.post(
        f"{BASE_URL}/transfer",
        params={"transfer": transfer, "account": account, "hmac": hmac_value}
    )

    print(f"\n  Server Response [{response.status_code}]:")
    print(f"  {response.json()}")

# Demo 2
def demo_tampered_request():
    print_separator("DEMO 2: Tampered Request (HMAC unchanged)")

    # Original values used to generate HMAC
    original_transfer = "5000"
```

```python
    account = "B"
    message = f"transfer={original_transfer}&account={account}"
    hmac_value = generate_hmac(message)

    # Attacker modifies transfer to 9000, but reuses the original HMAC
    tampered_transfer = "9000"

    print(f"  Original message  : {message}")
    print(f"  HMAC (for 5000)   : {hmac_value}")
    print(f"  Tampered transfer : {tampered_transfer} (HMAC not updated)")

    response = requests.post(
        f"{BASE_URL}/transfer",
        params={"transfer": tampered_transfer, "account": account, "hmac":
hmac_value}
    )

    print(f"\n  Server Response [{response.status_code}]:")
    print(f"  {response.json()}")


# Demo 3
def demo_no_hmac_endpoint():
    print_separator("DEMO 3: Tampered Request — Insecure Endpoint (No HMAC
check)")

    tampered_transfer = "9000"
    account = "B"

    print(f"  Sending tampered transfer={tampered_transfer} with NO HMAC to
insecure endpoint")

    response = requests.post(
        f"{BASE_URL}/transfer-no-hmac",
        params={"transfer": tampered_transfer, "account": account}
    )

    print(f"\n  Server Response [{response.status_code}]:")
    print(f"  {response.json()}")
    print("\n  Accepted blindly")


# Demo 4
def demo_local_hmac_generation():
```

```python
    print_separator("DEMO 4: Local HMAC Generation (no server needed)")

    test_cases = [
        ("transfer=5000&account=B", "5000 → B (original)"),
        ("transfer=7000&account=B", "7000 → B (modified amount)"),
        ("transfer=5000&account=C", "5000 → C (modified account)"),
    ]

    for message, label in test_cases:
        hmac_value = generate_hmac(message)
        print(f"\n  [{label}]")
        print(f"  Message : {message}")
        print(f"  HMAC    : {hmac_value}")


# Main

if __name__ == "__main__":
    print("\nHMAC Request Integrity Demo")
    print("   Shrihari V | 23BCI0083")

    demo_local_hmac_generation()

    print("\n\n─── Server-dependent demos (ensure server.py is running) ───")
    try:
        demo_legitimate_request()
        demo_tampered_request()
        demo_no_hmac_endpoint()
    except requests.exceptions.ConnectionError:
        print("\n Could not connect to server.")
        print("  Start the server first: python server.py")

    print("\n" + "=" * 60)
    print("  Demo complete.")
    print("=" * 60 + "\n")
```

5. Program Output

5.1 Local HMAC Generation & Legitimate / Tampered Requests

The output below shows the avalanche effect in HMAC — even a single character change in the message produces a completely different hash. Demo 1 confirms the legitimate request is accepted (200). Demo 2 confirms the tampered request is rejected (403) with the expected vs provided HMAC clearly shown.

```
(.venv) PS C:\Users\shrih\OneDrive\Programs\PycharmProjects\HMAC Verification> python client_demo.py

 HMAC Request Integrity Demo
    Shrihari V | 23BCI0083

 ============================================================
   DEMO 4: Local HMAC Generation (no server needed)
 ============================================================

   [5000 → B (original)]
   Message : transfer=5000&account=B
   HMAC    : 4081d94f1bdcfac0e3e9ca5e2d9536e7283c9eaa00bfeb8cba89632620e584c6

   [7000 → B (modified amount)]
   Message : transfer=7000&account=B
   HMAC    : a0d0b426cf14666d9d2ee1a66cf7f3cc7c864a0188da2d5a9c92c3b70ad1f5cf

   [5000 → C (modified account)]
   Message : transfer=5000&account=C
   HMAC    : 77bbe1cec8e0bb7a54a3eed1bc000a7cac7d47a1b5d7da23f3452839beb8057b

   Notice: Even a small change in the message produces a completely different HMAC.


 ── Server-dependent demos (ensure server.py is running) ──

 ============================================================
   Server Response [200]:
   {'data': {'account': 'B', 'transfer': '5000'}, 'message': 'Request accepted. HMAC verified — data integrity confirmed.', 'statu
 s': 'success'}

 ============================================================
   DEMO 2: Tampered Request (HMAC unchanged)
 ============================================================
   Original message  : transfer=5000&account=B
   HMAC (for 5000)   : 4081d94f1bdcfac0e3e9ca5e2d9536e7283c9eaa00bfeb8cba89632620e584c6
   Tampered transfer : 9000 (HMAC not updated)

   Server Response [403]:
   {'expected_hmac': '58afa210cd547f5418032864bff8608baf7c5ccd6cc845b1c6491deb4945e2d1', 'message': 'Request rejected. HMAC mismat
 ch — possible tampering detected.', 'provided_hmac': '4081d94f1bdcfac0e3e9ca5e2d9536e7283c9eaa00bfeb8cba89632620e584c6', 'status'
 : 'rejected'}
```

Figure 1 — HMAC generation, legitimate request (200 OK), tampered request (403 Rejected)

5.2 Insecure Endpoint Accepting Tampered Request

Demo 3 demonstrates that the insecure endpoint (/transfer-no-hmac) accepts the tampered value (transfer=9000) without any verification, returning 200 OK — highlighting the consequence of missing integrity checks.

```
============================================================
  DEMO 3: Tampered Request — Insecure Endpoint (No HMAC check)
============================================================
  Sending tampered transfer=9000 with NO HMAC to insecure endpoint

  Server Response [200]:
  Original message  : transfer=5000&account=B
  HMAC (for 5000)   : 4081d94f1bdcfac0e3e9ca5e2d9536e7283c9eaa00bfeb8cba89632620e584c6
  Tampered transfer : 9000 (HMAC not updated)

  Server Response [403]:
  {'expected_hmac': '58afa210cd547f5418032864bff8608baf7c5ccd6cc845b1c6491deb4945e2d1', 'message': 'Request rejected. HMAC mismat
ch — possible tampering detected.', 'provided_hmac': '4081d94f1bdcfac0e3e9ca5e2d9536e7283c9eaa00bfeb8cba89632620e584c6', 'status'
: 'rejected'}

============================================================
  DEMO 3: Tampered Request — Insecure Endpoint (No HMAC check)
============================================================
  Sending tampered transfer=9000 with NO HMAC to insecure endpoint

  Server Response [200]:
  Sending tampered transfer=9000 with NO HMAC to insecure endpoint

  Server Response [200]:
  {'data': {'account': 'B', 'transfer': '9000'}, 'message': 'Request accepted (NO integrity check — vulnerable endpoint).', 'stat
us': 'success'}

  Accepted blindly — demonstrates why integrity checks matter.

============================================================
  Demo complete.
============================================================
```

Figure 2 — Insecure endpoint accepts tampered request blindly (200 OK)

5.3 Server Log

The Flask server log confirms the three request outcomes: 200 for the legitimate request, 403 for the tampered HMAC request (highlighted in red by Flask), and 200 for the insecure endpoint.

```
(.venv) PS C:\Users\shrih\OneDrive\Programs\PycharmProjects\HMAC Verification> python server.py
Starting HMAC verification server on http://127.0.0.1:5000
Secret key in use: Shrihari_23BCI0083
 * Serving Flask app 'server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
Starting HMAC verification server on http://127.0.0.1:5000
Secret key in use: Shrihari_23BCI0083
 * Debugger is active!
 * Debugger PIN: 652-313-238
127.0.0.1 - - [10/Apr/2026 02:32:33] "POST /transfer?transfer=5000&account=B&hmac=4081d94f1bdcfac0e3e9ca5e2d9536e7283c9eaa00bfeb8cba
89632620e584c6 HTTP/1.1" 200 -
127.0.0.1 - - [10/Apr/2026 02:32:33] "POST /transfer?transfer=9000&account=B&hmac=4081d94f1bdcfac0e3e9ca5e2d9536e7283c9eaa00bfeb8cba
89632620e584c6 HTTP/1.1" 403 -
127.0.0.1 - - [10/Apr/2026 02:32:33] "POST /transfer-no-hmac?transfer=9000&account=B HTTP/1.1" 200 -
```

Figure 3 — Flask server log showing 200, 403, and 200 responses

6. How HMAC Protects Integrity

```
Client side:
  message  = "transfer=5000&account=B"
  hmac_val = HMAC-SHA256(message, secret_key)
  sends → transfer=5000 & account=B & hmac=<hmac_val>


Server side:
```

```
  recomputes = HMAC-SHA256(f"transfer={t}&account={acc}", secret_key)
  if recomputed == provided_hmac → ACCEPT
  else                           → REJECT (403)

Attacker modifies transfer=5000 → transfer=9000
  Their HMAC is still for 5000 → mismatch detected
  Cannot forge correct HMAC without knowing the secret key
```

Key implementation note: hmac.compare_digest() is used instead of == to prevent timing-based side-channel attacks, where an attacker could infer HMAC bytes by measuring response times.

7. Results Summary

Scenario	Endpoint	Result
Legitimate request with valid HMAC	/transfer	200 — Accepted
Tampered request, HMAC unchanged	/transfer	403 — Rejected
Tampered request, no integrity check	/transfer-no-hmac	200 — Accepted blindly

8. Conclusion

The demonstration confirms that HTTP parameters can be freely modified when no integrity protection is in place. By incorporating HMAC-SHA256 into request verification, servers can reliably detect tampered data and reject malicious requests. The use of a shared secret key ensures that an attacker who intercepts the request cannot forge a valid HMAC without knowledge of the key, making HMAC an effective and lightweight mechanism for protecting sensitive operations such as financial transactions.